Facade

Season: 2 · Structural · Category: GoF · Level: Beginner · Build with Waqar

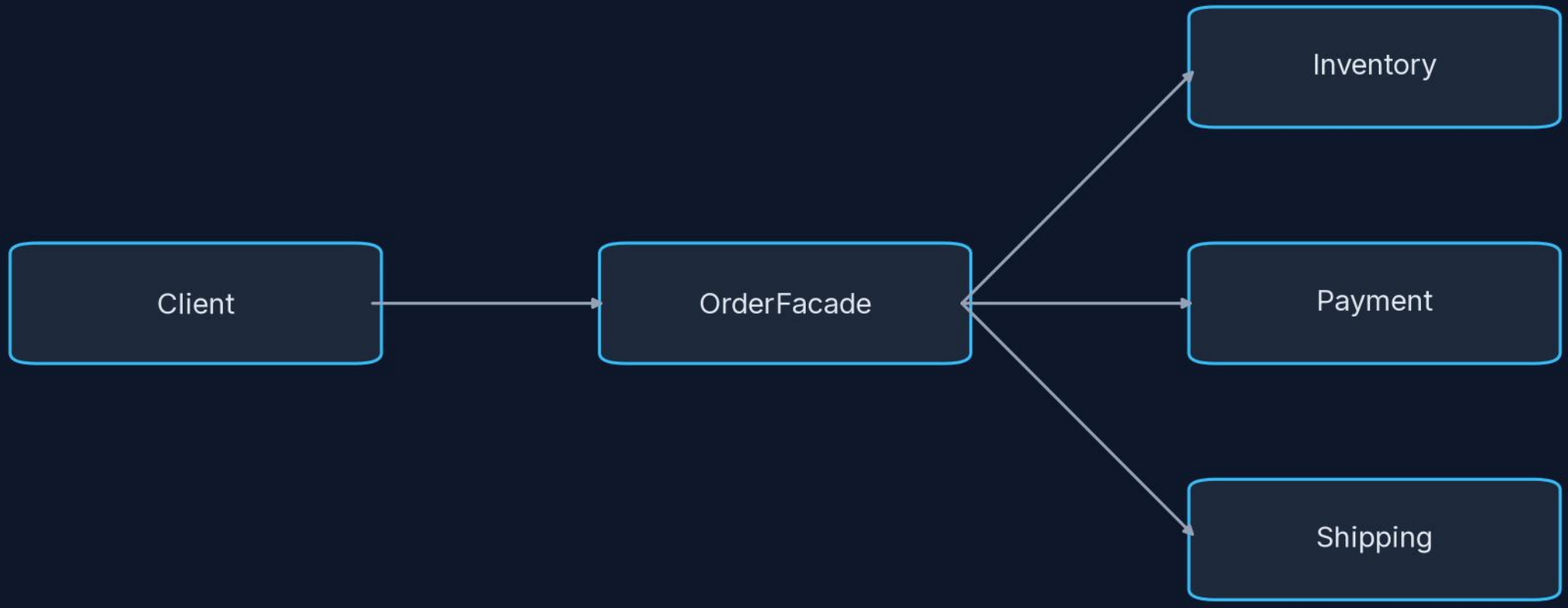
Today's Hook

- Today: Facade as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Facade structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Facade
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/facade/OrderFacade.java
package com.buildwithwaqar.patterns.gof.structural.facade;

final class Inventory { boolean reserve(String sku, int qty) { return qty > 0 && !sku.isBlank(); } }
final class PaymentGateway { boolean charge(String card, int cents) { return card != null && cents > 0; } }
final class Shipping { String ship(String sku) { return "TRACK-" + sku; } }

/** Simple interface over a noisy subsystem. */
public final class OrderFacade {
    private final Inventory inventory = new Inventory();
    private final PaymentGateway payments = new PaymentGateway();
    private final Shipping shipping = new Shipping();

    public String placeOrder(String sku, int qty, String card, int cents) {
        if (!inventory.reserve(sku, qty)) return "REJECTED:inventory";
        if (!payments.charge(card, cents)) return "REJECTED:payment";
        return "OK:" + shipping.ship(sku);
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.structural.facade;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Facade walkthrough ===");
    var facade = new OrderFacade();
    System.out.println(facade.placeOrder("sku-42", 1, "4111", 1999));
    System.out.println(facade.placeOrder("", 1, "4111", 1999));
    System.out.println(facade.placeOrder("sku-42", 1, "4111", 0));
  }
}
```

Walkthrough Output

```
=== Facade walkthrough ===
OK:TRACK-sku-42
REJECTED:inventory
REJECTED:payment
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Facade
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Facade → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.